Week 7: Refactoring and Design Patterns

Messy Code

```
system_name = "WUA SSRS"

def submit_request(student, description):
    request = {
        "student": student,
        "description": description,
        "status": "Submitted"
    }
    print("Request saved")


def update_status(request, new_status):
    request["status"] = new_status
    print("Status changed")
    print("Send notification to student")
```

What is wrong with it?

- Configuration is just a global variable.
- Request creation is mixed with other application logic and has no central validation.
- Status updating is directly tied to notification behaviour.
- The code becomes harder to extend as new features are added.

Python implementation

```
from dataclasses import dataclass

from datetime import datetime

from uuid import uuid4
```

```
# 1. SINGLETON PATTERN
class SystemConfig:
    _instance = None
    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            cls._instance.system_name = "WUA SSRS"
        return cls._instance
# 2. FACTORY PATTERN
@dataclass
class ServiceRequest:
    reference: str
    student_id: str
    category: str
    description: str
    status: str
    created_at: str
class ServiceRequestFactory:
    @staticmethod
    def create(student_id, category, description):
        if not student_id.strip():
            raise ValueError("Student ID is required.")
        if not category.strip():
            raise ValueError("Category is required.")
        if not description.strip():
            raise ValueError("Description is required.")
        return ServiceRequest(
            reference=f"REQ-{uuid4().hex[:8].upper()}",
            student_id=student_id,
            category=category,
            description=description,
            status="Submitted",
            created_at=datetime.now().isoformat(
                timespec="seconds"
```

```
            ),
        )
# 3. OBSERVER PATTERN
class RequestStatusNotifier:
    def __init__(self):
        self._observers = []
    def subscribe(self, observer):
        self._observers.append(observer)
    def notify(self, request, old_status, new_status):
        for observer in self._observers:
            observer(request, old_status, new_status)
def print_status_update(request, old_status, new_status):
    print(
        f"Request {request.reference}: "
        f"{old_status} -> {new_status}"
    )
# Demonstration
if __name__ == "__main__":
    config1 = SystemConfig()
    config2 = SystemConfig()
    print("Same config instance:", config1 is config2)
    print("System:", config1.system_name)
    request = ServiceRequestFactory.create(
        student_id="STU001",
        category="Maintenance",
        description="A classroom light is not working."
    )
    print("Created request:", request)
    notifier = RequestStatusNotifier()
    notifier.subscribe(print_status_update)
    old_status = request.status
    request.status = "In Progress"
    notifier.notify(request, old_status, request.status)
```

Code component	Pattern	Purpose
SystemConfig	Singleton	Reuses one configuration instance.
ServiceRequestFactory	Factory	Creates service request objects in one place.
RequestStatusNotifier	Observer	Notifies listeners when a request status changes.

Refactoring and Design Patterns Reflection

During Week 7, I applied the Singleton, Factory and Observer design patterns to the proposed WUA Student Support and Service Request System (SSRS).

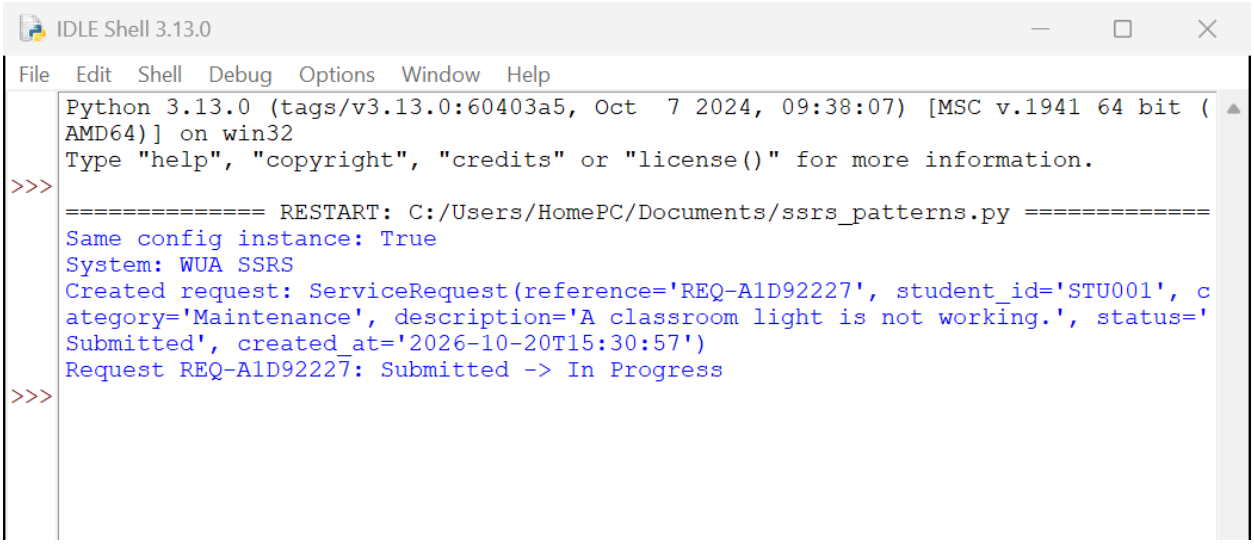
The Singleton pattern was used in the SystemConfig class to provide one shared configuration instance. This avoids creating multiple independent configuration objects. However, Singleton can introduce global-state and testing difficulties if used excessively.

The Factory pattern was implemented through ServiceRequestFactory. It centralises service request creation and validates essential fields before creating an object. This reduces duplicated creation logic and makes the process easier to maintain.

The Observer pattern was demonstrated using RequestStatusNotifier. When a service request's status changes, registered observers receive the old and new status values. This separates the status-change process from the actions triggered by that change, making it easier to add further listeners, such as an email notification service.

Overall, these patterns demonstrate how refactoring can improve code organisation, maintainability and extensibility. Nevertheless, design patterns should only be used where they solve a genuine design problem. The current implementation is a prototype and requires further testing, persistent data storage and appropriate security controls before it could be used in a real university environment.

Output


IDLE Shell 3.13.0
File Edit Shell Debug Options Window Help
Python 3.13.0 (tags/v3.13.0:60403a5, Oct 7 2024, 09:38:07) [MSC v.1941 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
========== RESTART: C:/Users/HomePC/Documents/ssrs_patterns.py ==========
Same config instance: True
System: WUA SSRS
Created request: ServiceRequest(reference='REQ-A1D92227', student_id='STU001', c
ategory='Maintenance', description='A classroom light is not working.', status='
Submitted', created_at='2026-10-20T15:30:57')
Request REQ-A1D92227: Submitted -> In Progress
>>>